Lecture 12: DPO, RAG, Scaling Laws

Andrei Arsene Simion

Columbia University

December 2, 2025

Outline

- Supervised Fine Tuning and DPO
- RAG
- Scaling Laws

Outline

- Supervised Fine Tuning and DPO
- RAG
- Scaling Laws

What happens if you ask an LLM a question?

- You train a LLM
- What next?

To start, let us consider how a language model at the end of autoregressive pretraining answers a query. A strong model, Llama 3.1 405B, will continue from "The president of the united states in 2006 was":

> George W. Bush, the governor of Florida in 2006 was Jeb Bush, and John McCain was an Arizona senator in 2006 - who later lost to obama. September 1 – U.S. President Bush signs an executive order to provide more options for faith-based organizations when delivering social services that are paid for with federal funds. January 1 – The current deadline set by United States Congress in the Unlawful Internet Gambling Enforcement Act, aka UIGEA. search: Amendments to the 1961 International Convention for the Protection of New Varieties of Plants require plant breeders' rights include farmer's privilege. 2009 was a common year starting on Thursday of the Gregorian calendar, the 2009th year of the Common Era (ce) and Anno Domini (ad) designations, the 9th year of the 3rd millennium, the 9th year of the 21st century, and the 10th and last year of the 2000s decade.

What happens if you ask an aligned LLM a question?

- How would you align an LLM?

Continuing the above example, a model trained with RLHF (and other post-training tools), would answer the question very differently. Asking Tülu 3 405B the same question "The president of the united states in 2006 was," is says concisely:

> George W. Bush was the president of the United States in 2006. He served two terms in office, from January 20, 2001, to January 20, 2009.

Supervised Fine Tuning

- So, like in HW but at a much bigger scale, we can build a Transformer based (Decoder) Language Model
- Given another question question like "Tell me who the president is?", how might a LLM like GPT3 answer? It would babble on
- We can leave GPT3 alone and see what it generates, or, we can give it examples like the above and have it "understand" what to do given such a prompt
- Specifically, we aim to give the model a prompt like "You are a helpful chatbot. Question: Tell me who the president is. Answer: " and have it generate " Donald Trump".
- Specifically, we can have a loss on "Donald Trump STOP" and feed it "START Donald Trump" ... Note the start token and the stop token STOP
- Note, if we do not do this, and just have the model generate text, it might just generate the questions again. Or the answer, or whatever (we saw this with another example)

Supervised Fine Tuning Schematic

- Generally, the task here is very similar to what we saw for classification before
- With classification, you want to get a score from an input sentence
- Here, you input text and you want the model to generate text that answers your question

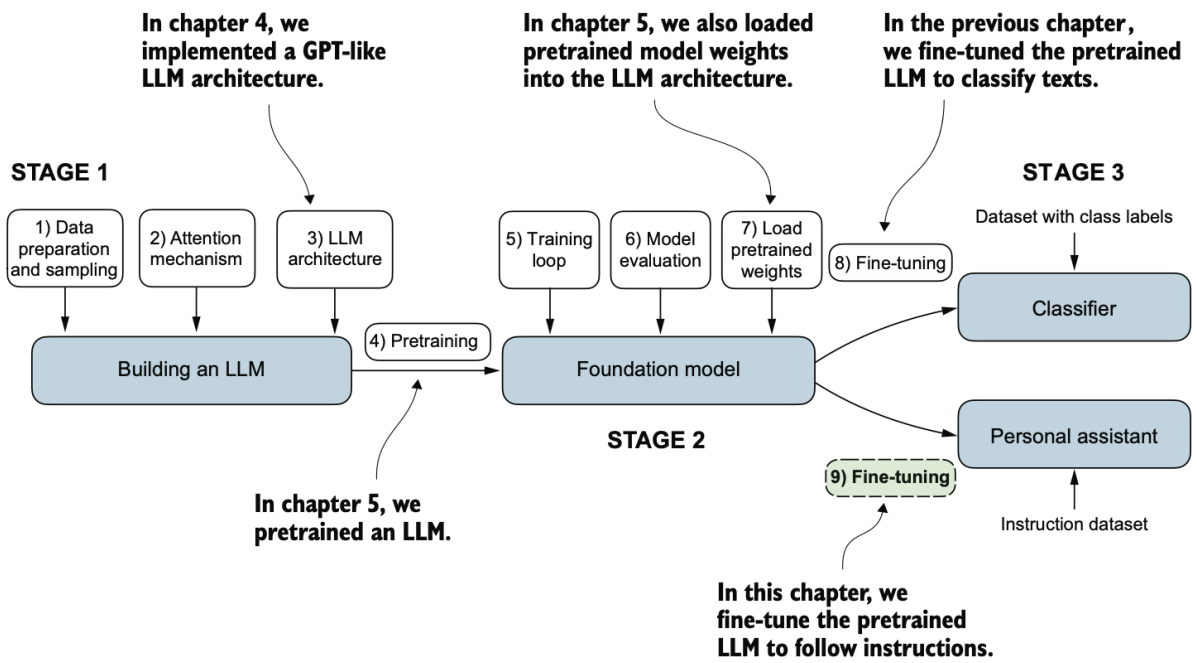

Figure 7.1 The three main stages of coding an LLM. This chapter focuses on step 9 of stage 3: fine-tuning a pretrained LLM to follow human instructions.

Supervised Fine Tuning Examples

- Instruction tuning: Examples

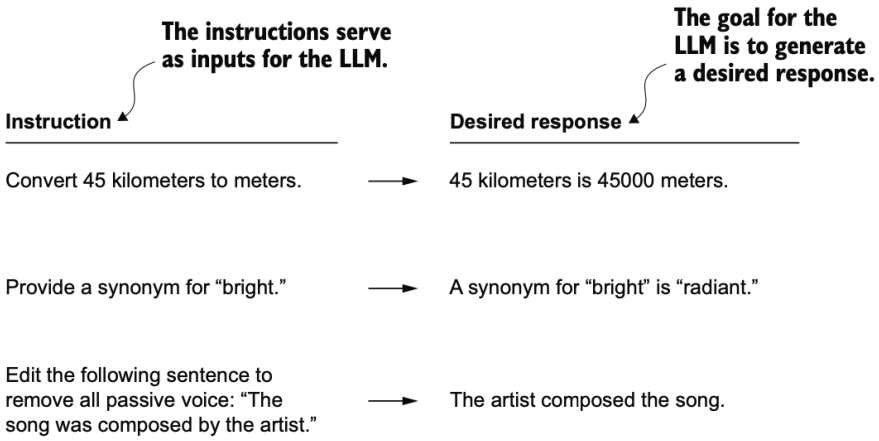

Figure 7.2 Examples of instructions that are processed by an LLM to generate desired responses

Supervised Fine Tuning inputs can differ

- Different models have different setups / prompts

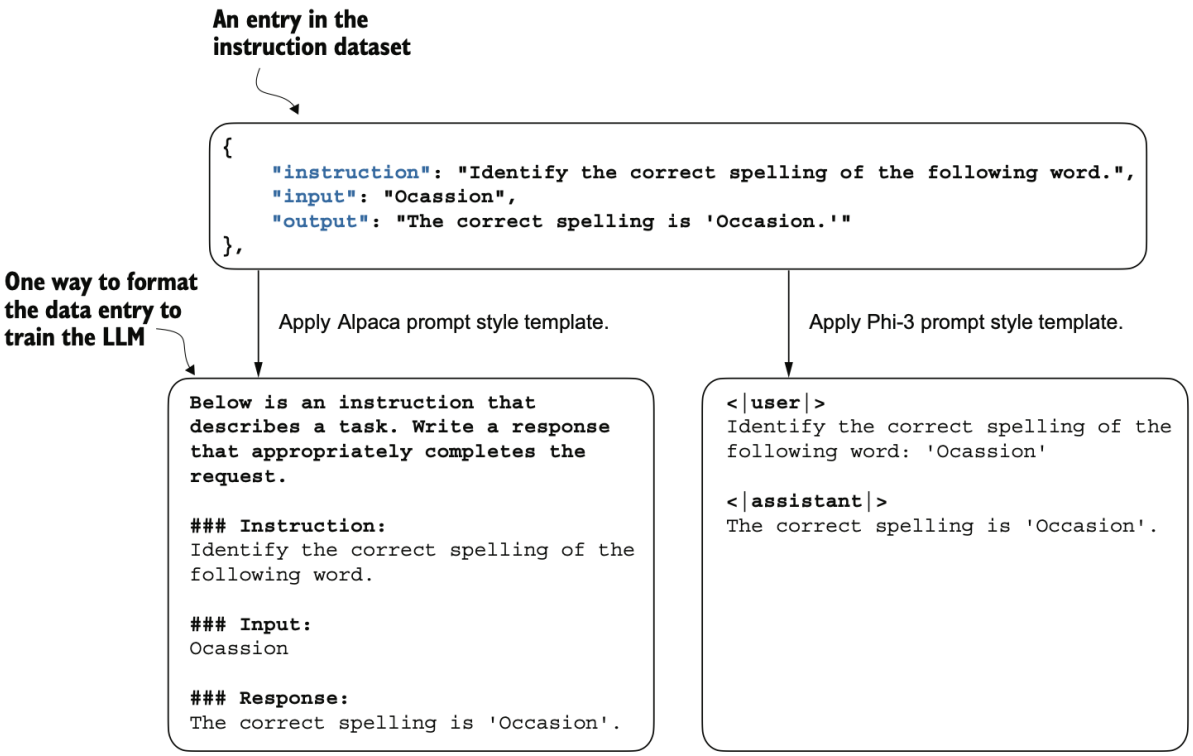

Figure 7.4 Comparison of prompt styles for instruction fine-tuning in LLMs. The Alpaca style (left) uses a structured format with defined sections for instruction, input, and response, while the Phi-3 style (right) employs a simpler format with designated <|user|> and <|assistant|> tokens.

Supervised Fine Tuning Step By Step

- A graphic of how it works, in PyTorch; just the expected response is in the loss and we remove extra STOP / PAD tokens by using -100 to not include them in the loss

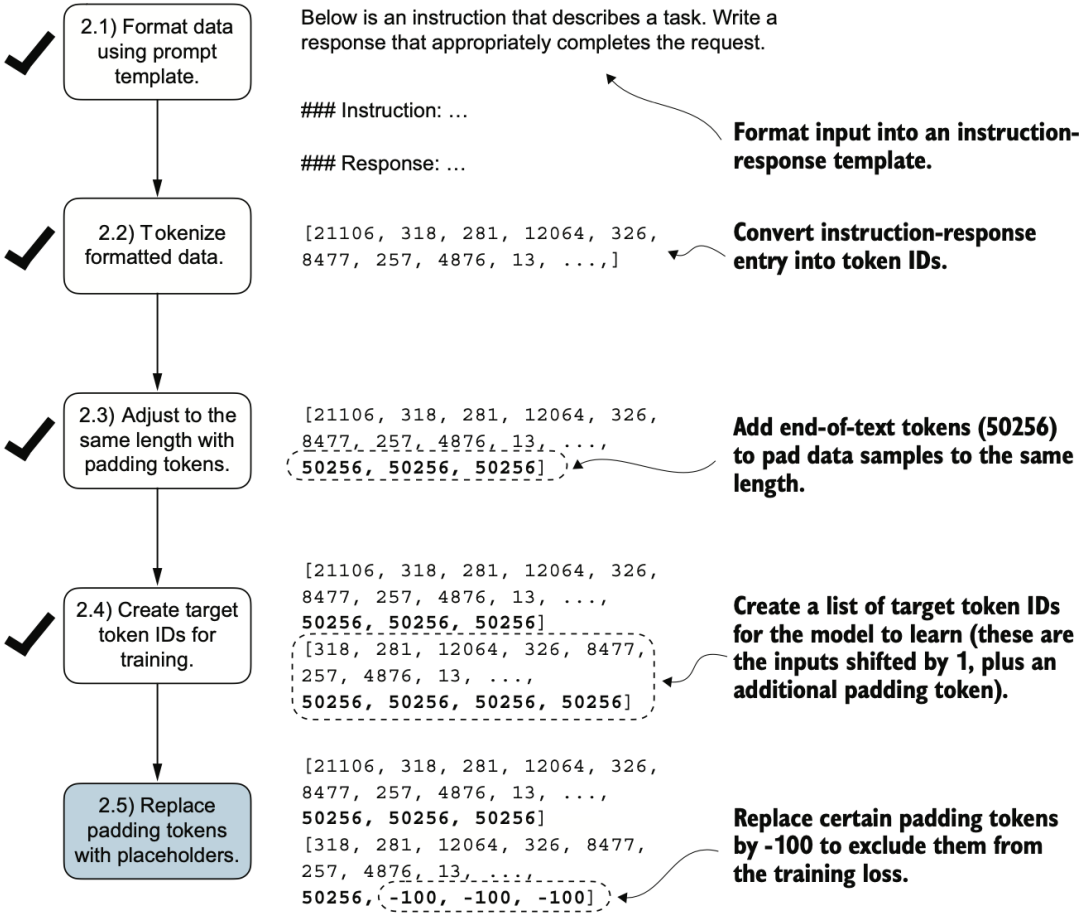

Figure 7.11 The five substeps involved in implementing the batching process. After creating the target sequence by shifting token IDs one position to the right and appending an end-of-text token, in step 2.5, we replace the end-of-text padding tokens with a placeholder value (`-100`).

Example

- Suppose we have two examples $(x_1^{(1)}, x_2^{(1)}, y_1^{(1)}, y_2^{(1)})$ and $(x_1^{(2)}, x_2^{(2)}, x_3^{(2)}, y_1^{(2)}, y_2^{(2)}, y_3^{(2)})$
- Example 1
  - $x = (x_1^{(1)}, x_2^{(1)}, y_1^{(1)}, y_2^{(1)})$
  - $y = (x_2^{(1)}, y_1^{(1)}, y_2^{(1)}, PAD)$
- Example 2
  - $x = (x_1^{(2)}, x_2^{(2)}, x_3^{(2)}, y_1^{(2)}, y_2^{(2)}, y_3^{(2)})$
  - $y = (x_2^{(2)}, x_3^{(2)}, y_1^{(2)}, y_2^{(2)}, y_3^{(2)}, PAD)$
- But if we need to put this in a batch, this needs to become 2 by 6 so we need to make the first example
  - $x = (x_1^{(1)}, x_2^{(1)}, y_1^{(1)}, y_2^{(1)}, PAD, PAD)$
  - $y = (x_2^{(1)}, y_1^{(1)}, y_2^{(1)}, PAD, -100, -100)$
- Example 1: 3 loss terms
- Example 2: 4 loss terms

Supervised Fine Tuning Output

- Once we train the model, we need to evaluate the quality of the generation vs a known response
- Your model might be small and specific (spell corrector), and can user a large LLM for this

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:

Rewrite the sentence using a simile.

Input:

The car is very fast.

Correct response:

>> The car is as fast as lightning.

Model response:

>> The car is as fast as a bullet.

Getting a Personal Assistant

- Is SFT enough?
- Using SFT there are a few key limitations:
  - Data reflects human outputs, not preferences
    - You're training the model to mimic what a human might say, but not to choose among many plausible options based on what humans prefer
  - What about ambiguity and multiple good answers
    - Often, there's more than one good answer to a prompt
    - SFT only shows one per example, which may bias the model away from equally good or better answers
  - SFT doesn't capture what people like vs what's merely correct
    - Just because something is factually right doesn't mean it's helpful, kind, clear, or engaging
    - SFT doesn't optimize for human satisfaction or alignment with human values

Getting a Personal Assistant

- For example, suppose we have an LLM and we've fine tuned it to generally add two numbers
- ... Suppose we want to build an assistant to help us do calculations and understand how to do them ... We have a question: "What is 2 + 2*3?"
- We might get two generations like below:
  - Generation 1: "8"
  - Generation 2: "First, I want to use the laws of operations to calculate 2*3 and get 6. Then, I add 2 and get 8."
- Neither of the above is wrong, but we want to entice the model to generate like the second answer ... How might we do that?
- So, to review, to get a personal assistant we need a few things ...

Step 1: Build a general LLM

- For this part, we will just train a Transformer for next token prediction
- To get this working, you process data so that generally $(y_0, \ldots, y_{t-1})$ is used as the input and $(y_1, \ldots, y_t)$ is used as the output
- The above is generally now a Transformer decoder, but it can be any model; Transformers are used because you can scale and they do very well
  - For the above you still need to determine the right data to use
  - You need quality data ...
  - You still need to know the number of parameters vs your compute budget
  - You need to carefully do the optimization ...
  - Etc.
- The idea of this step is we need a general model that understand language well

Step 2: Fine tune to your use case

- Here, the idea is we want a model that understands that given a format of examples it should turn around and answer specifically the questions we address it
- At this step, we take examples $(x, y_0, \ldots, y_{t-1})$ as input and have targets $(y_1, \ldots, y_t)$
- Examples at this step are like the ones we saw before in the previous slides

Step 3: Refine the model - Reinforcement Learning!

- Here, we want to have the model generally output y_w over y_l given x
- In this case, y_w and y_l might be somewhat correct but given x as input y_w is deemed better and we want to entice the model to generate y_w
  - Many applications for this ...
  - Addition: tell is a step-by-step answer
  - Block the model: $x =$"How do I build a bomb?" ... Don't answer!

Step 3: Refine the model

- How might we do this step?
- Suppose we have a reward $r^*(x, y)$ which we give to the model for generating y given x - this is the Human reward which we do not know
  - Good generations get high rewards, bad generations get low rewards
- Suppose $\pi^*(y|x)$ is the optimal model and $\pi_{ref}(y|x)$ is the model we have after SFT
- Suppose we have data (x, y_w, y_l) where y_w is preferred to y_l as completions given x

Bradley-Terry Model for rewards

- We can assume that human preference is written as

$$P^*(y_w > y_l|x) = \frac{e^{r^*(x,y_w)}}{e^{r^*(x,y_w)} + e^{r^*(x,y_l)}} = \sigma(r^*(x,y_w) - r^*(x,y_l))$$

  - The above is the Bradley-Terry model which connects the probability of $y_w > y_l$ with the optimal reward r^*
  - We do not know r^*, we can estimate it as r_ϕ or just see what r^* yields if we do some math (more later)
- Goal: I want to get $\pi_\theta(y|x)$ which puts more probability on y_w than y_l ... This is the same model as $\pi_{ref}(y|x)$ but modified / aligned with human feedback for this task

Getting to $\pi_\theta \to \pi^*$

- How might we start the process of getting π^*
- To start, initialize $\pi_\theta(y|x) = \pi_{ref}(y|x)$ and let's modify π_θ to go towards π^* ...
- Suppose $x \sim D$, the space of all prefixes
- We want an objective that rewards us for generating what we want but we also want to not move too far away from the reference model $\pi_{ref}(y|x)$
- Why? Well, we still want to be a language model; we don't want to "game" the reward
- What would help us not move too far from $\pi_{ref}(y|x)$?
- Recall the KL Divergence $KL[p||q] = \sum_y p(y) \log \frac{p(y)}{q(y)}$

KL Divergence for any two distributions p and q

- Recall the KL Divergence

$$KL[p||q] = \sum_y p(y) \log \frac{p(y)}{q(y)} = E_p[\log \frac{p(y)}{q(y)}]$$

- We have log is concave so generally by Jensen's nequality

$$E[\log X] \leq E[\log X]$$

  for any random variable X
- The above is just $\log[(a+b)/2] \geq \frac{\log a + \log b}{2}$ but more generally
- We have

$$KL[p||q] = \sum_y p(y) \log \frac{p(y)}{q(y)} = E_p[\log \frac{p(y)}{q(y)}] = -E_p[\log \frac{q(y)}{p(y)}]$$

KL Divergence for any two distributions p and q

We have, by Jensen on $f(x) = -\log x$:

$$KL = -E_p[\log \frac{q(y)}{p(y)}] \geq -\log E_p[\frac{q(y)}{p(y)}] = -\log \sum_y q(y) = -\log 1 = 0$$

- So, KL Divergence is a positive metric
- It can be shown that KL is zero if an only if p and q are the same
- Generally, we can use the KL Divergence as a type of "regularizer" to make sure one distribution is "close" to some base distribution

Back to Step 3; a Reinforcement Learning objective

Consider the objective

$$J(\theta) = \max_{\theta, x \sim D, y \sim \pi_\theta(y|x)} E[r^*(x,y)] - \beta KL[\pi_\theta(y|x) || \pi_{ref}(y|x)]$$

- Typically, problems like this are done with Reinforcement Learning
  - We have a reward
  - We have a policy π
  - We want to get the optimal policy which gives us the best reward
- What is the above trying to do?
- $\beta > 0$, with larger values making it harder and harder to move away from $\pi_{ref}(y|x)$
- Given an output, we try and figure out θ so that the language model we are working with maximizes the reward and it does not veer too far from the reference we start with $\pi_{ref}(y|x)$
- Notice that we don't know the analytic form of the reward $r^*(x,y)$ ... What to do?

Reinforcement Learning

Consider the objective

$$J(\theta) = \max_{\theta, x \sim D, y \sim \pi(y|x)} E[r^*(x,y)] - \beta KL[\pi(y|x)||\pi_{ref}(y|x)]$$

- So the above objective is typically dealt with via Reinforcement Learning
- ... This is a standard objective in the space where we want to maximize the reward and find a best policy $\pi(y|x)$
- Here, you replace r^* with a reward empirical estimator r_ϕ and solve the optimization via REINFORCE or PPO
- This is difficult to do, and leads to a complicated optimization which needs to be carefully done
- The outcome of this work is a updated language model which is modifies to get close to the rewards you give it for different y
  - In the context of RL, the language model is the policy we seek to find and there is a way to express everything in (state, action, reward) time series as usual

Reward Modeling; How do you get r_ϕ for RL?

- How do we find out r_ϕ?
- Well if we have

$$P^*(y_w > y_l|x) = \frac{e^{r^*(x,y_w)}}{e^{r^*(x,y_w)} + e^{r^*(x,y_l)}} = \sigma(r^*(x,y_w) - r^*(x,y_l))$$

and if we have data $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$ we can

- Take $\pi_{ref}(y|x)$ and add some parameters on top ϕ so that we modify the architecture slightly to get a scalar for any y
- Use the fact that we want to maximize $P(y_w > y_l|x)$ and consider the objective

$$\frac{-\sum_{i=1}^N \log P(y_w^{(i)} > y_l^{(i)}|x^{(i)})}{N} = -E_{(x,y_w,y_l)}[\log \sigma(r_\phi(x,y_w) - r_\phi(x,y_l))]$$

- We can do Back-propagation on the above
- Ultimately, we figure out ϕ so we can build the reward function ...

Reinforcement Learning

Consider the objective with r_ϕ

$$J(\theta) = \max_{\theta, x \sim D, y \sim \pi(y|x)} E[r_\phi(x, y)] - \beta KL[\pi(y|x) || \pi_{ref}(y|x)]$$

- The objective above was introduced in a paper InstructGPT which introduced RLHF, Reinforcement Learning from Human Feedback
- To get the reward, we need $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$ and some optimization
- Once we have the reward, we can impute it and get data $\{(x^{(i)}, y^{(i)}, r_\phi(x^{(i)}, y^{(i)})\}_{i=1}^N$ and we try and optimize $J(\theta)$ on this data to arrive at π_θ, whatever is optimal for the reward
- Crucially, notice that we build the reward r_ϕ first, so this is several steps

RLHF

Below is the standard Reinforcement Learning from Human Feedback (RLHF) loop

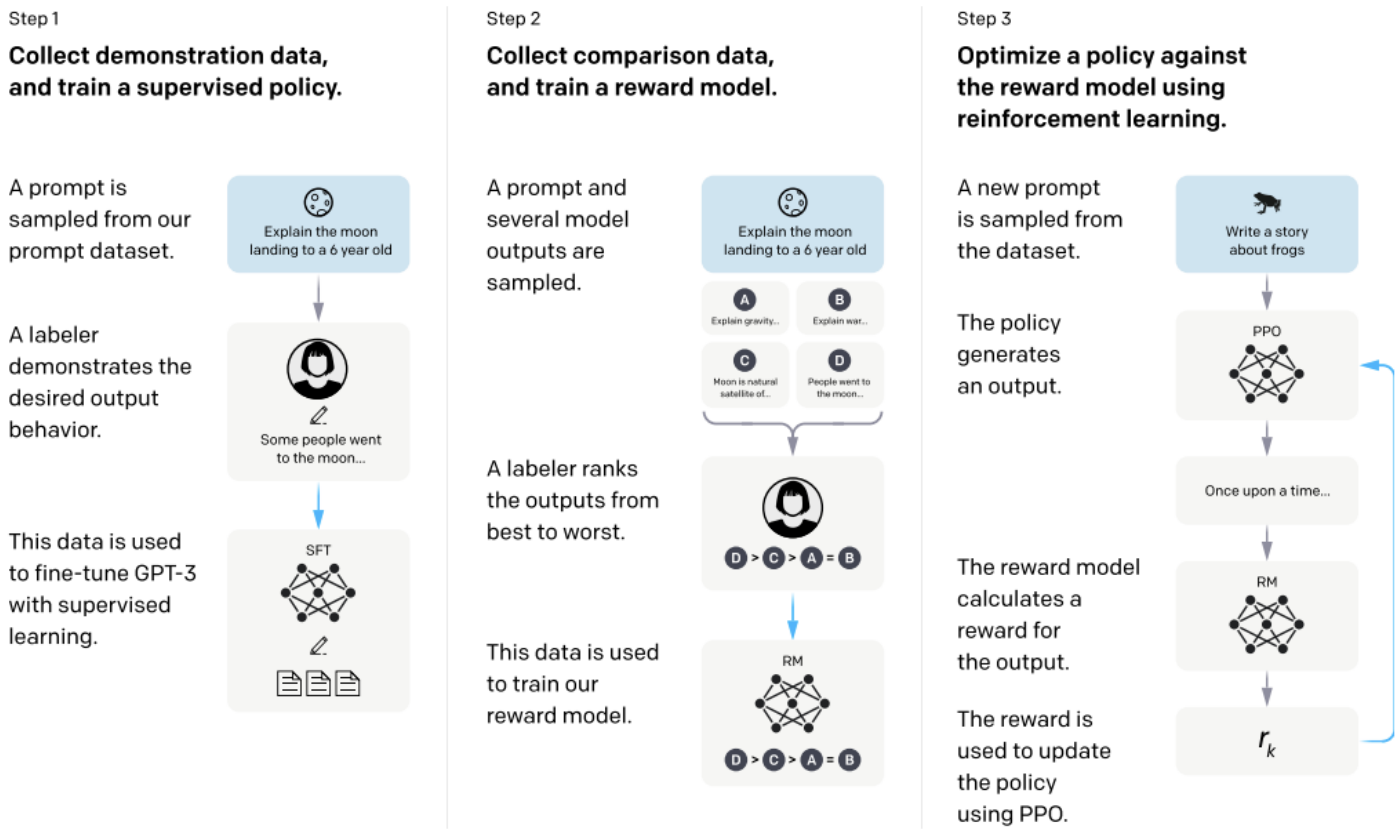

Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

DPO: Direct Preference Optimization

Left part: RLHF
Right part: alignment without Reinforcement Learning (DPO)

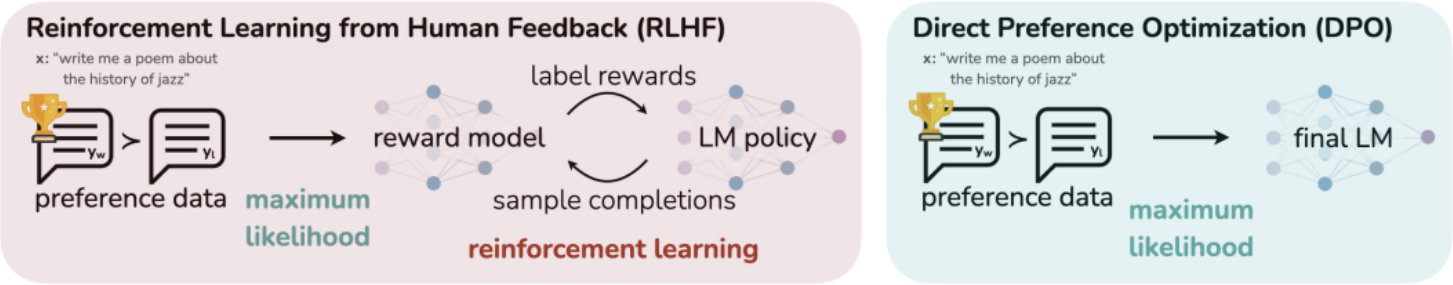

Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

Back to the objective $J(\theta)$

Consider the original objective ... Let's simplify it slowly

$$J(\theta) = \max_{\theta, x\sim D, y\sim\pi(y|x)} E[r^*(x,y)] - \beta KL[\pi(y|x)||\pi_{ref}(y|x)]$$

$$\begin{aligned} J(\theta) &= \max_{\pi} E_{x\sim D}[E_{y\sim\pi(y|x)}[r^*(x,y) - \beta KL[\pi(y|x)||\pi_{ref}(y|x)]] \\ &= \max_{\pi} E_{x\sim D}[E_{y\sim\pi(y|x)}[r^*(x,y) - \log \frac{\pi(y|x)}{\pi_{ref}(y|x)}] \\ &= \min_{\pi} E_{x\sim D}[E_{y\sim\pi(y|x)}[\log \frac{\pi(y|x)}{[\frac{1}{Z(x)}\pi_{ref}(y|x)\exp\frac{r^*(x,y)}{\beta}]} - \log Z(x)] \end{aligned}$$

In the above we've set the normalization constant

$$Z(x) = \sum_y \pi_{ref}(y|x)\exp\frac{r^*(x,y)}{\beta}$$

Back to Step 3

- In the above we have the normalization constant

$$Z(x) = \sum_y \pi_{ref}(y|x) \exp \frac{r^*(x,y)}{\beta}$$

- This is not very useful, as it is an exponential sum
- However we have the probability distribution

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{ref}(y|x) \exp \frac{r^*(x,y)}{\beta}$$

- Using this, we then get a simplification ...

Simplification

Consider again the original objective and the simplifications we made ...

$$J(\theta) = \max_{\theta, x \sim D, y \sim \pi(y|x)} E[r^*(x,y)] - \beta KL[\pi(y|x)||\pi_{ref}(y|x)]$$

$$\begin{aligned} J(\theta) &= \min_{\pi} E_{x \sim D}[E_{y \sim \pi(y|x)}[\log \frac{\pi(y|x)}{\pi^*(y|x)}] - \log Z(x)] \\ &= \min_{\pi} E_{x \sim D}[KL[\pi(y|x)||\pi^*(y|x)] - \log Z(x)] \end{aligned}$$

- Since $Z(x)$ does not depend on π, the minimization above is attained when the KL term is zero which happens when $\pi = \pi^*$
- So, using the math above the optimal policy we get from our objective is π^*

From r^* to the π^*

- We have the optimal policy is expressed directly as

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{ref}(y|x)\exp\frac{r^*(x,y)}{\beta}$$

- This gives

$$r^*(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{ref}(y|x)} + \beta\log Z(x)$$

- So, we have a correspondence between π^* and r^* and if we find one we find the other
- We had the assumption that

$$P^*(y_w > y_l|x) = \frac{e^{r^*(x,y_w)}}{e^{r^*(x,y_w)} + e^{r^*(x,y_l)}}$$

Replacing r^* by π^*

- We had the assumption that

$$P^*(y_w > y_l|x) = \frac{e^{r^*(x,y_w)}}{e^{r^*(x,y_w)} + e^{r^*(x,y_l)}} = \sigma(r^*(x,y_w) - r^*(x,y_l))$$

- If we plug in

$$r^*(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$$

we get

$$P^*(y_w > y_l|x) = \sigma(\beta \log \frac{\pi^*(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{ref}(y_l|x)})$$

What does this mean

- We can go back to the probabilities of y_w vs y_l but now we can solve for π not r_ϕ
- We get

$$P^*(y_w > y_l|x) = \sigma(\beta \log \frac{\pi^*(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{ref}(y_l|x)})$$

- What does this mean? For the above, we have that to modify our model so that it tends to generate y_w as opposed to y_l, we should minimize a loss like

$$-\log P(y_w > y_l|x)$$

- We can have many pairs of data $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N$ and in that case we can focus on minimizing

$$-\frac{\sum_{i=1}^N \log P(y_w^{(i)} > y_l^{(i)}|x^{(i)})}{N}$$

- Here, we are maximizing the the Expected log probability of y_w getting a higher score than y_l and this implies a model π^*

What does this mean

- In summary, DPO assumes we have data $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N$ and optimizes

$$-\frac{\sum_{i=1}^N \log P(y_w^{(i)} > y_l^{(i)} | x^{(i)})}{N}$$

- This is the same as

$$\mathcal{L}(\theta) = -\frac{\sum_{i=1}^N \log \sigma(\beta \log \frac{\pi_\theta(y_w^{(i)}|x^{(i)})}{\pi_{ref}(y_w^{(i)}|x^{(i)})} - \beta \log \frac{\pi_\theta(y_l^{(i)}|x^{(i)})}{\pi_{ref}(y_l^{(i)}|x^{(i)})})}{N}$$

- At optimality, we hope to converge to π^* having started at π_θ

DPO

- The above model is called Direct Preference Optimization
- At a high level, this is a Reinforcement Learning alternative which has no modeled reward function
- This encourages the model to assign higher probability to preferred outputs — no explicit reward function is needed, but the loss function implicitly reflects preference
- DPO uses human preference data directly to optimize a contrastive loss between a preferred output and a less preferred one
- ... You could say the log-probabilities of preferred vs dispreferred outputs act as a proxy reward
- DPO is effectively matching the behavior of an implicit reward function that ranks outputs based on preference

DPO Caveats with Implementation

- To implement DPO, are there any caveats?
- Let's say you needed to align a LLAMA 70B model in this way
- You need to work with the probabilities

$$\sigma(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)})$$

  in the objective
- You need 2 types of logits: for the model you change are for the model that is a reference
- To get this to work right, you might want to cache the reference model logits ... This way, you don't need to do forward passes through it over and over and you don't need to load this model
- Issues like this come up in practice

DPO Experiments

Here, GPT4 is used as a judge of the two methods

Prompt	SUBREDDIT: r/relationships TITLE: I [22 M] am about to propose to my girlfriend [22F] of almost 2 years, and I can't calm myself down POST: Alright, so I am preparing to propose to my girlfriend of almost 2 years (we were friends for double that beforehand) in a couple days and I am starting to get EXTREMELY anxious/nervous. We have talked about this for months, and I know she will say yes, but I can't seem to relax. The other day I bought the ring, which I fell in love with myself and know she will too. And then two days ago, I asked her dad's blessing. Everything has gone great so far, but now leading up to it I can't think about anything else. I can't sleep. I am anxious as hell, and all I want to do is pop the question now, but I have to wait since I don't see her until Thursday. I know how I am going to do it. I have thoroughly planned it out and I am more than prepared. But, at the same time, I am freaking out. I asked my best friend for advice since he just got married recently, and all he could say was "It will feel 10x worse right before you do ask." Any suggestions on how to relax leading up to it? I would like to be able to sleep...
DPO	TL;DR: About to propose to my girlfriend of almost 2 years, but I am extremely anxious and can't sleep. How can I relax?
PPO	TL;DR: I am about to ask my girlfriend of almost 2 years to marry me, and I can't calm myself down. Any tips on how to relax? I would like to be able to sleep tonight (and the night before).....
Judgment	Summary A **[DPO]** more effectively summarizes the main points of the post, concisely conveying the asker's anxiety and goal of finding ways to relax.

Table 5: Sample summaries to a post from the TL;DR test set. **DPO** sample generated with a temperature 0.25; **PPO** sample generated at a temperature of 0. The order in which summaries are presented in randomized when evaluating with GPT-4, so the order in the Judgment may not correspond to the order in the table. For clarity, post-hoc annotations are included in bold, formatted as **[annotation]**. These annotations are not part of the model generations.

DPO Issues?

Issues with DPO?

- Assumption Mismatch .. DPO is derived from a probabilistic model assuming humans make decisions based on a softmax over reward values. But real human preferences are often noisy and inconsistent. If the assumptions don't hold, the model can misinterpret the preference data
- Implicit Reward Model ... DPO skips the explicit reward model used in RLHF and optimizes policy directly. This is great for simplicity but means you can't inspect, analyze, or reuse a separate reward model, which can be valuable for understanding model behavior or for multi-stage training setups ... I can't really see the reward even thought I know it is connected to r^*
- Sensitivity to Preference Quality ... DPO uses pairwise comparisons (e.g., A > B) to guide training. If those preferences are sparse, inconsistent, or biased, the optimization can go astray. There's no reinforcement loop to "pull back" the model like in PPO (used in RLHF)
- Less Control Over Output Distribution ... Because DPO skips reinforcement-style feedback loops, it may offer less control over certain safety and behavioral constraints that can be more finely tuned with RLHF (e.g., toxicity reduction via explicit reward shaping)
- Lack of Exploration ... In RLHF (especially with PPO), the agent is encouraged to explore and learn from feedback. DPO, being a supervised-style method, does not explicitly explore alternative outputs—it just learns from the comparisons you give it

Why use RLHF?

Benefits of RLHF?

- Explicit Reward Modeling ... RLHF gives you a separate reward model, which can be valuable for:
  - Auditing what the model "likes"
  - Reusing it across models
  - Improving interpretability
- Fine-Grained Control via Reward Tuning ... Want your model to be funny and safe and informative? RLHF lets you weight these reward components explicitly and dynamically adjust the mix ... There is no "optimal" implicit reward
- Better Performance in Complex or High-Stakes Domains ... In cases where getting nuanced behavior is critical (like legal or medical advice), RLHF might better capture and enforce complex preference structures (+ rewards)
- Proven Track Record ... Many high-profile models (like ChatGPT and Claude) use RLHF because it's a mature and battle-tested method. If you're aiming for production-level robustness, RLHF is still the gold standard

Is there anything other than DPO?

Alternatives to DPO

- There are a few DPO alternatives ... Ask Chat GPT for some papers!
- Big idea is to change some of the final optimization but sometimes there is no nice analytic derivation ...

Outline

- Supervised Fine Tuning
- RAG
- Scaling Laws

Retrieval Augmented Generation

- Given a model like GPT3 trained up to 2023, how do we answer the question "Who is the current president?"
- Given this question, the LLM will give an answer but the answer (because of data cutoff) would be "Joe Biden"
- This is an example of Hallucination; in this case the model gives back a confident answer but it is wrong because its data is not fresh enough
- Retraining the model is very expensive and fine tuning for language modeling might break some old information
- Moreover, the model weights might not even be available (GPT3, GPT4 etc)

RAG

- Step 1: Add new data chunks to a document store, and embed each of these chunks as vectors (use a BiEncoder like SentenceBERT)
- Step 2: When a query comes in, get the closest semantic chunks
- Step 3: When you generate an answer, add those chunks to the prompt thereby making the model aware of the current state of things
  - Bring in new information
  - Bring in information the model is not aware of
  - Increase the precision of output

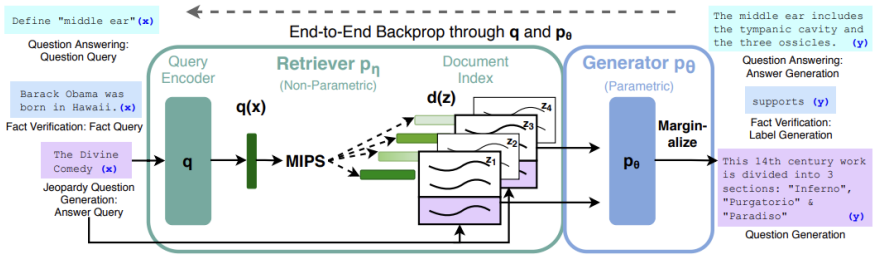

Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x, we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i. For final prediction y, we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

RAG - Another Example

- Another example: a question about Open AI which is not in the model's training set (a fact?)
- Some ideas to consider
  - What do you embed? Documents? Chunks?
  - Do you use metadata?
  - How do you chunk?

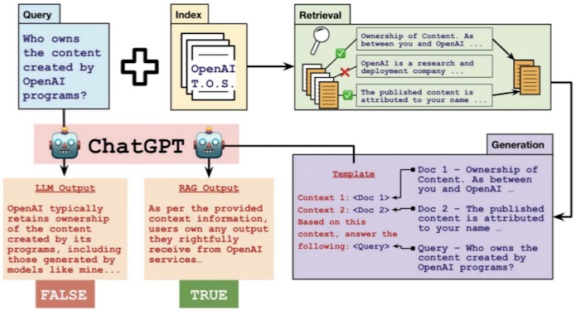

Fig. 7.2: Practical illustration of the RAG workflow, using a question about the ownership of OpenAI output as an example. If we ask ChatGPT the question, we get the wrong answer. Instead, a RAG call with a vector index of the OpenAI terms of service gives the correct answer.

RAG

- To retrieve chunks, you can use a BiEncoder
- Offline, documents are chunked and the chunks are embedded by the BiEncoder and stored in a database
- At runtime the query is encoded and the chunks with the highest inner product against the query are retrieved
  - For a query, you get the documents which have the highest cosine similarity in the embedding space
  - Using brute force, this is $O(N)$
  - However, we can do this fast in $O(\log N)$
- Ranking can be tuned ... For example, to get chunks that are temporally fresh at the top, you can rank newer chunks before older chunks by boosting using a factor

RAG - Chunking Analysis / Strategy is its own niche field

- What to do?
  - Tune the chunk length, but remember each chunk gets an embedding and only finitely any tokens can go into an LLM to get an embedding
  - Pick a specific piece of text in the document and then use that as the query
  - Q - Do you get a subset of this text as the closest chunk C? Can also have metrics using Levenshtein distance:
    - Recall $= \frac{|Q| - LD(C,D)}{|Q|}$
    - Precision $= \frac{|C| - LD(C,Q)}{|C|}$
- Furthermore, we can also do *Semantic* chunking, whereby we (for example) split on sentences and then merge greedily sentences that are close to each other
  - We can also use NSP from BERT to say C_1 and C_2 should be merged?
  - Can we have a custom BiEncoder model? How do we determine the cutoff score to merge two chunks?
- Either of the above is possible and it matters ...

Outline

- Supervised Fine Tuning and DPO
- RAG
- Scaling Laws

Scaling Laws

- After 2020, Transformer style Language Models kept getting bigger and bigger
  - GPT1, GPT2 and GPT3 were each essentially the same architecture but they were different in the total parameter count and the amount of data they were trained on
  - Other models by other teams beyond Open AI were introduced ... For example, Alphabet released Gopher
- Question: Given a compute budget, how many parameters should the model have and how much data should it be trained on
- Optimal here means lowest Training ($\sim$ Test) perplexity or loss $\mathcal{L}$

Table 1 | **Current LLMs**. We show five of the current largest dense transformer models, their size, and the number of training tokens. Other than LaMDA (Thoppilan et al., 2022), most models are trained for approximately 300 billion tokens. We introduce *Chinchilla*, a substantially smaller model, trained for much longer than 300B tokens.

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Scaling Laws are everywhere

- Roughly speaking, a scaling law connects the Error / Loss with the size of the model of the size of the data we have
- Let's do an example
- Suppose we have a distribution with constant variance σ^2 and are given data $\{x_1, \ldots, x_N\}$ and want to find out an estimator for the mean μ
- Typically, we can use $\hat{\mu} = \frac{x_1 + \ldots + x_N}{N}$ to estimate μ

Scaling Laws are everywhere

- The Mean Squared Error (MSE) between the estimator and the true parameter is

$$MSE = E((\hat{\mu} - \mu)^2)$$

- We have $E(\hat{\mu}) = \mu$ so that $Bias = 0$ and because this is unbiased we have

$$E((\hat{\mu} - \mu)^2) = Bias^2 + Var(\hat{\mu}) = Var(\hat{\mu})$$

- Simplifying, we get

$$MSE = E((\hat{\mu} - \mu)^2) = \frac{\sigma^2}{N}$$

- Form here, we get

$$\log(MSE) = 2\log\sigma - \log N$$

- This is a Scaling Law: We are connecting the *Error* with the size of the data set we have, N

Scaling Laws are everywhere

- The above tells us a lot
- Specifically, given we know that the log expected Error is for size 1000, we can use the above to figure out what the error would be for size $N = 1$ M
- In this case, it is easy to figure this out directly, but this is not always the case so figuring out such a "best fit line" is crucial

Chinchilla Vs Other Models

- What about for LLMs?
- At a high level, the authors establish empirical rules which connect the Error or Loss with the model parameter count and the training dataset size
- Specifically, having established the true line, Chinchilla is trained using the same approximate budget (= parameters × tokens) as Gopher and GPT-3 but notice that it has much fewer parameters and does much better on downstream tasks
- Why does this matter?
  - Smaller models have less inference latency
  - Smaller models are easier to load and use
  - Smaller models are "easier to train"; we just need lots of data but less GPUs
  - Also, it is nice to understand when a model is "baked" ... When should we stop training a model?

Table 1 | **Current LLMs**. We show five of the current largest dense transformer models, their size, and the number of training tokens. Other than LaMDA (Thoppilan et al., 2022), most models are trained for approximately 300 billion tokens. We introduce *Chinchilla*, a substantially smaller model, trained for much longer than 300B tokens.

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Scaling Laws

- Key question: What is compute cost?
- Compute Cost = FLOPs
- FLOPs: Floating Point Operations per second
- Sort of like big O notation, used to measure work
- Adding two floats costs 1
- Multiplying two floats costs 1
- What else?

FLOPs Calculations for Matrices

- Suppose we have two vectors a and b of dimension d
- How many FLOPs do we need to compute $a^\intercal b$?
- $a^\intercal b = \sum_{i=1}^{d} a_i b_i$
  - We need d multiplications
  - We need $d - 1$ additions
  - So, we need about $2d - 1$ FLOPs to compute an inner product
- What if we have two matrices A and B of dimensions $m \times d$ and $d \times n$
- What is the compute cost of AB
  - Each row of A has dimension d and each column of B has d elements
  - So, for the first column of B with all rows of A we need $m(2d - 1)$ FLOPs since there are m rows
  - But B has n columns
  - So, we need approximately $mn(2d - 1)$ FLOPs
- How else can we use this?

Scaling Laws - Forward Pass and Backward Pass FLOPs

- Suppose we have a Transformer
- Suppose we train it on D tokens and suppose it has N total parameters
- A Neural Network is a bunch of matrices ...
- Computations like the previous ones can give us several results
- For derivations, see the References to this lecture and the Chinchilla Appendix
  - Generally, 2 * Forward FLOPs $\sim$ Backward FLOPs
  - Total FLOPs for Forward Pass in the Transformer: $\sim 2ND$
  - Total FLOPs for Backward Pass in the Transformer: $\sim 4ND$
  - Total FLOPs $\sim 6ND$

FLOPs for an MLP

- Below is a small MLP
- For this MLP, the author calculates the total number of FLOPs needed for a forward and backward pass per layer
- You see that the Backward FLOPs is about twice the Forward FLOPs

	Direction	Op	FLOPs
0	fprop	linear	1233125376
1	fprop	relu	4096
2	fprop	linear	1048576
3	fprop	relu	128
4	fprop	linear	2560
5	fprop	cross_entropy	0
6	fprop	backward	0
7	bprop	cross_entropy	0
8	bprop	linear	2560
9	bprop	linear	2560
10	bprop	relu	128
11	bprop	linear	1048576
12	bprop	linear	1048576
13	bprop	relu	4096
14	bprop	linear	1233125376
15	bprop	add_	1234184980

We can see that the first layer (red) has the same flop count for forward and backward pass while the other layers (blue, green) have a ratio of 2:1. The final weight update (yellow) is 2x the number of parameters of the network.

Optimizing N and D over FLOPs

- So, in terms of FLOPS work, the Backward Pass has approximately twice the number of FLOPs as the Forward Pass
- Given a Transformer Language Model model with N parameters and trained on D tokens, we can estimate the number of FLOPs needed as $6ND$
- A goal, therefore, might be to find

$$N^*(C), D^*(C) = argmin_{\{FLOPs(N,D)=C\}} \mathcal{L}(N, D)$$

- In the above, $\mathcal{L}$ is the (Test) loss from the model and C is the compute budget, which is the total number of FLOPs

Optimization Problem and Implications

- We want to fix C and solve

$$N^*(C), D^*(C) = argmin_{\{FLOPs(N,D)=C\}} \mathcal{L}(N, D)$$

- This is a very hard non analytical problem. At a high level, the authors do this purely empirically by varying N and D and looking at the loss when $FLOPs(N, D) = C$
- We have optimality in this setting for $(N^* D^*)$ if $F(N_*, D_*) = C$ and

$$\mathcal{L}(N_*, D_*) < \mathcal{L}(N, D) \; \forall N, D, FLOPs(N, D) = C$$

- From this, we see that given this compute budget C, we should train a model with N_* parameters on D_* tokens vs anything else
- Is there a relationship between N_*, D_* and C? Can we find a rule to get N_* and D_* given C?

Experiment 1: Fix model size N, train each for various number of tokens D

- For the first experiment the authors do the following
  - They fix models of different sizes, considering a range of model from 70 M to 10 B total parameters N
  - For each model, train using 4 different setting of tokens D
  - As we train longer, we get different compute budgets
- Question: Given a compute budget, which model gets the lowest loss?

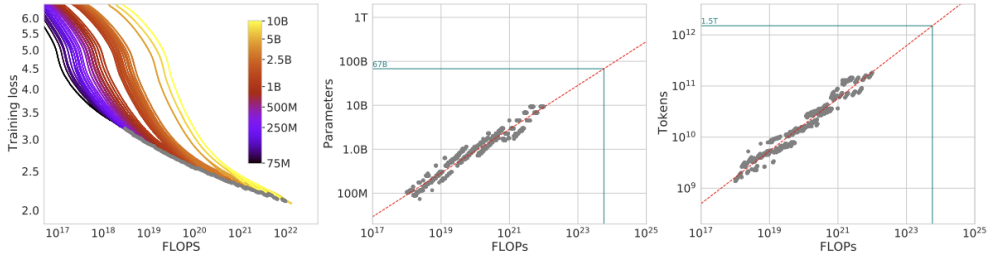

Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Experiment 1: Fix model size N, train each for various number of tokens D

- Look at the left plot, fixing an element on the FLOPS $x = C$ axis and going vertically up
- The best model is the one that gets the lowest loss - it is compute optimal as all models on this vertical have the same FLOPs but different (N, D)
- This is the grey envelope in the picture on the left
- Insights: A $10B$ parameter model is not optimal for 10^{19} FLOPs, but it is optimal for 10^{22}
- Given $C = 10^{22}$ and $N^* = 10B$, we can infer D^* since $6ND = C$

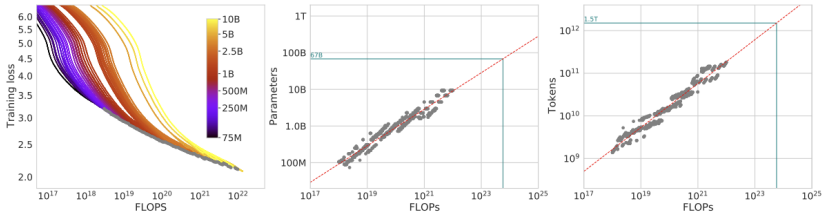

Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Experiment 1: Fix model size N, train each for various number of tokens D

- On the middle and right plots, the authors take the best model for a given plot and either plot C vs N^* or C vs D^*
- At a high level, what they observe are the scaling laws: $N^* \sim C^{1/2}$ and similarly $D^* \sim C^{1/2}$
- For example, if your compute budget is $C = 10^{20}$, then your optimal parameter count is $N^* \sim 1B$ and you'd ideally train for $D^* \sim 10^{10}$
- Moreover, the authors observe that $D^* = 20N^*$, so each parameter should be trained on about 20 tokens

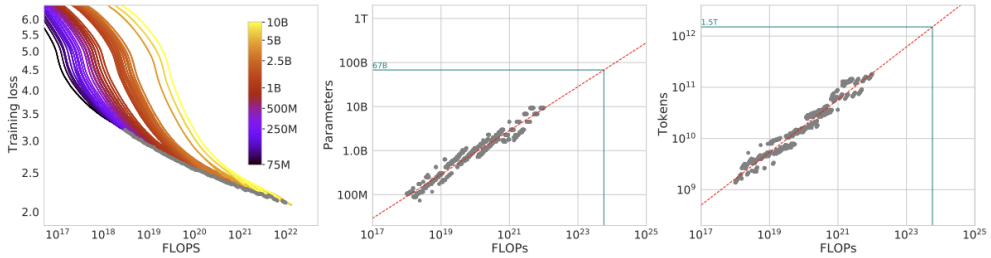

Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Experiment 2: Given a FLOPs budget C, find the best model pair (N^*, D^*)

- The above is one experimental setup, the authors do another to make sure the rules they find are plausible
  - Here, they select FLOPs C as being from 6×10^{18} to 3×10^{21}
  - For each budget, they train several models of various sizes (N, D)
  - Note that, once you pick the parameters N and the FLOPs C, you effectively set the D since $ND \sim C$
  - The optimal model for a given FLOPs C budget is the one that gave us the lowest loss
  - Graphically, this is the vertex of each parabola
- For example, if we have 6×10^{18} FLOPs then the ideal model size is around $N = 200$ M

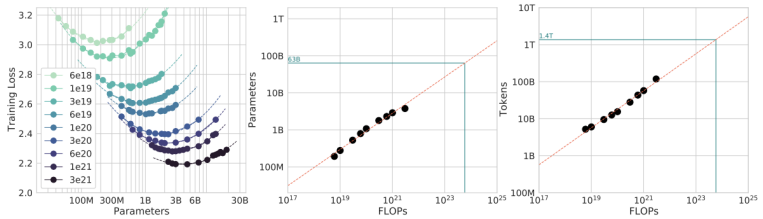

Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Experiment 2: Given a FLOPs budget C, find the best model pair (N^*, D^*)

- Given a FLOPs budget, the authors extract N^* and infer D^*
- As before, they plot $\log C$ against $\log N^*$ and observe that $N^* \sim C^{1/2}$ and similarly $D^* \sim C^{1/2}$
- For Gopher, a 280 B parameter model trained on 300 B tokens, it seems you should have selected $N^* \sim 70$ B and $D^* \sim 1.4$ T tokens

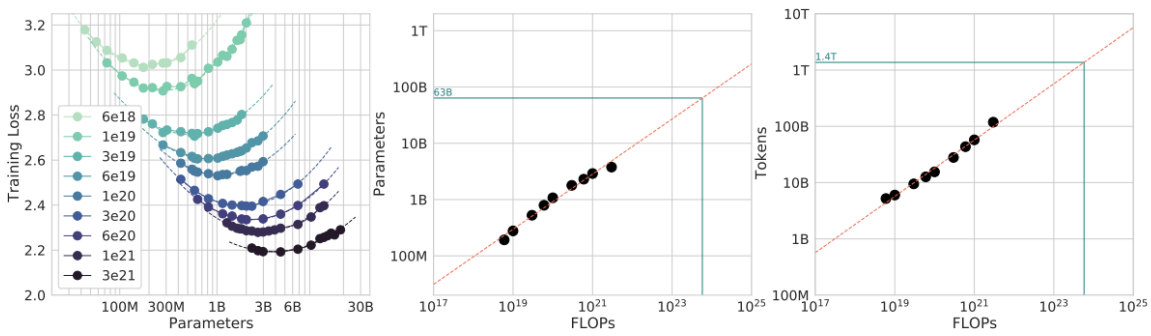

Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Experiment 3: Analytic formula for the Loss

- Another experiment the authors do is they theorize that the Training loss can be given by an analytical formula, for example
$$\hat{\mathcal{L}}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$
- The above has nothing to do with FLOPs, it is a theoretical decomposition (see Appendix)
- From Approaches 1 and 2, they take the data and then fit this model
- After optimization, they get $E = 1.69$, $A = 406.6$, $B = 410.7$, $B = 410.7$, $\alpha = 0.34$ and $\beta = 0.28$
- This is a very nasty optimization, and they need to carefully initialize and tune their optimizer

Experiment 3: Analytic formula for the Loss

- Now that you have the loss, you can fix the FLOPs and see which model gave the best loss and that N and D were
- Given a fixed FLOPs budget, what is the $N^*(C)$ and $D^*(C)$?
- They look at contours of this function, and get $N^*(C) \sim C^{1/2}$ and $D^*(C) \sim C^{1/2}$ yet again

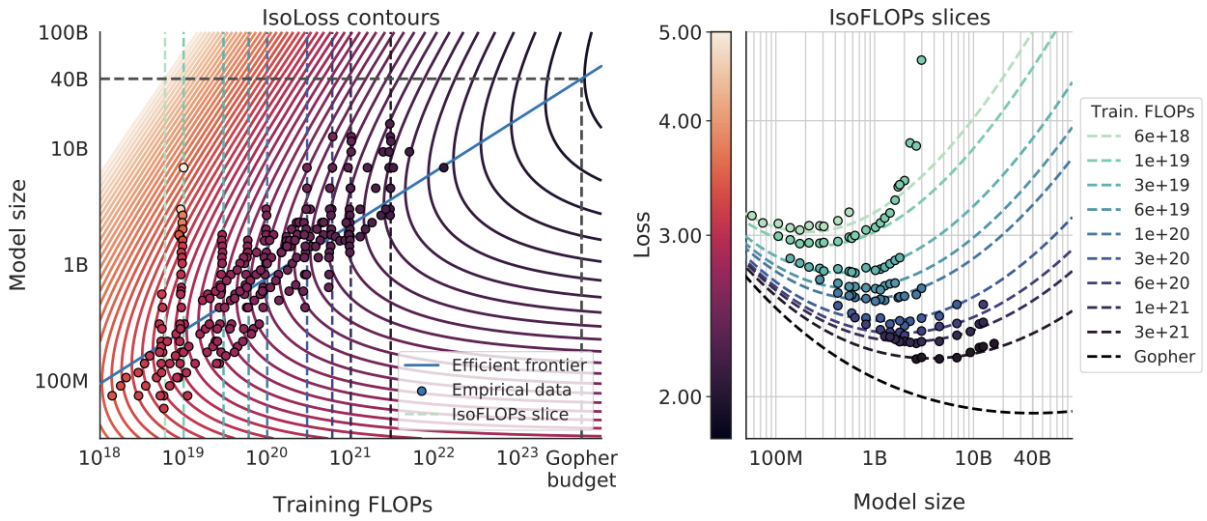

Figure 4 | **Parametric fit.** We fit a parametric modelling of the loss $\hat{L}(N, D)$ and display contour (**left**) and isoFLOP slices (**right**). For each isoFLOP slice, we include a corresponding dashed line in the left plot. In the left plot, we show the efficient frontier in blue, which is a line in log-log space. Specifically, the curve goes through each iso-loss contour at the point with the fewest FLOPs. We project the optimal model size given the *Gopher* FLOP budget to be 40B parameters.

Approach 2 vs Approach 3: Some variation

- The rule $N^*, D^* \sim C^{1/2}$ is not quite that, there is a bit of variation between the approaches
- However, the orders of magnitude are right and this is much better than before when there was zero guidance

For Approaches 2 and 3, we show the estimated model size and number of training tokens for a variety of compute budgets in Table A3. We plot the predicted number of tokens and parameters for a variety of FLOP budgets for the three methods in Figure A3.

	Approach 2		Approach 3	
Parameters	FLOPs	Tokens	FLOPs	Tokens
400 Million	1.84e+19	7.7 Billion	2.21e+19	9.2 Billion
1 Billion	1.20e+20	20.0 Billion	1.62e+20	27.1 Billion
10 Billion	1.32e+22	219.5 Billion	2.46e+22	410.1 Billion
67 Billion	6.88e+23	1.7 Trillion	1.71e+24	4.1 Trillion
175 Billion	4.54e+24	4.3 Trillion	1.26e+24	12.0 Trillion
280 Billion	1.18e+25	7.1 Trillion	3.52e+25	20.1 Trillion
520 Billion	4.19e+25	13.4 Trillion	1.36e+26	43.5 Trillion
1 Trillion	1.59e+26	26.5 Trillion	5.65e+26	94.1 Trillion
10 Trillion	1.75e+28	292.0 Trillion	8.55e+28	1425.5 Trillion

Where are other models vs the Scaling laws?

- We can see that many bigger models are under trained
- Specifically, before 2022, many models have too many parameters and too little training data

.

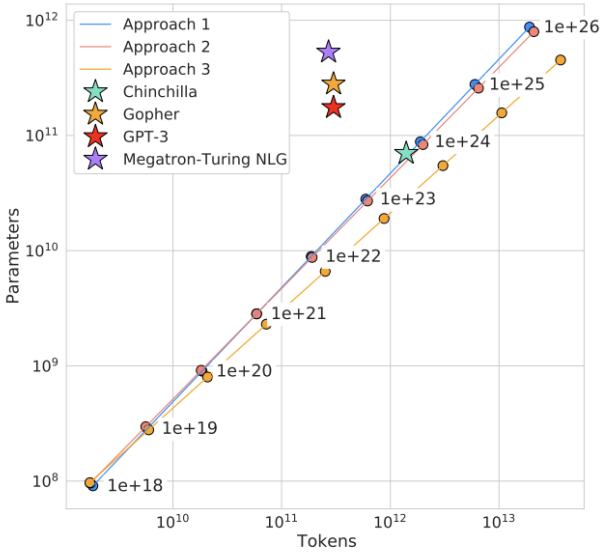

Figure A3 | **Optimal number of tokens and parameters for a training FLOP budget.** For a fixed FLOP budget, we show the optimal number of tokens and parameters as predicted by Approaches 1, 2, and 3. For an alternate representation, see Figure 1.

What about BERT?

- Recall BERT Large was 340 M parameters
- It was trained on BooksCorpus (800 M words) and English Wikipedia (2500 M words)
- Each word has about 1.25 tokens after tokenization, so we have about 4.125 B unique tokens
- BERT trained for 1 M steps with batch size 256 and each row had 512 tokens
- So BERT Large was trained on 131 B tokens but only 4.125 B of them were unique
- BERT went over the same tokens roughly 30 times

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7
$\text{XLNet}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	94.0/87.8	88.4	94.4
+ additional data	126GB	2K	500K	94.5/88.8	89.8	95.6

Table 4: Development set results for RoBERTa as we pretrain over more data (16GB → 160GB of text) and pretrain for longer (100K → 300K → 500K steps). Each row accumulates improvements from the rows above. RoBERTa matches the architecture and training objective of $\text{BERT}_{\text{LARGE}}$. Results for $\text{BERT}_{\text{LARGE}}$ and $\text{XLNet}_{\text{LARGE}}$ are from Devlin et al. (2019) and Yang et al. (2019), respectively. Complete results on all GLUE tasks can be found in the Appendix.

What about ROBERTA?

- If Chinchilla holds best on unique tokens then we want $20 * 340M \sim 6.8$ B tokens, so BERT was trained too long on too few unique tokens
- One of the nuances of the Chinchilla paper is that the unique token sequences are the ones pushing your model up the most ... If you keep going too much over the same tokens you don't get much
- ROBERTA trained on more data (160 GB vs 16 GB so about 30 B tokens) and introduced dynamic masking and other optimization tricks so that repeated data was used better
- Also, it trained for far longer, thereby getting better performance

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7
$\text{XLNet}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	94.0/87.8	88.4	94.4
+ additional data	126GB	2K	500K	94.5/88.8	89.8	95.6

Table 4: Development set results for RoBERTa as we pretrain over more data (16GB → 160GB of text) and pretrain for longer (100K → 300K → 500K steps). Each row accumulates improvements from the rows above. RoBERTa matches the architecture and training objective of $\text{BERT}_{\text{LARGE}}$. Results for $\text{BERT}_{\text{LARGE}}$ and $\text{XLNet}_{\text{LARGE}}$ are from Devlin et al. (2019) and Yang et al. (2019), respectively. Complete results on all GLUE tasks can be found in the Appendix.

Chinchilla vs Gopher on Language Modeling

- Vs Gopher, Chinchilla does much better on all Language Modeling tasks
- This is not always the case since lowest Loss does not mean best downstream, but generally there is a positive correlation here

4.2.1. Language modelling

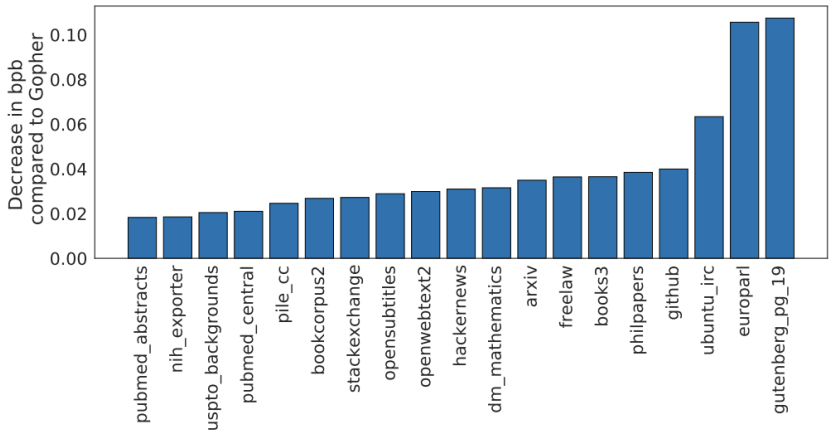

Figure 5 | **Pile Evaluation.** For the different evaluation sets in The Pile (Gao et al., 2020), we show the bits-per-byte (bpb) improvement (decrease) of *Chinchilla* compared to *Gopher*. On all subsets, *Chinchilla* outperforms *Gopher*.

Chinchilla vs Gopher on MMLU tasks

- Vs Gopher, a much larger model the compute-optimal Chinchilla does a lot better even on indirect tasks
- This type of behavior was present across many tasks
- So - you don't need just a larger model, you need a well trained one!
- Specifically, if I have a model w N parameters, do I just keep going over and over the same data and making gains? No, I need new data. And No, at some point even new data will not be optimal, for that FLOPs budget I should train a larger model on an appropriate amount of data

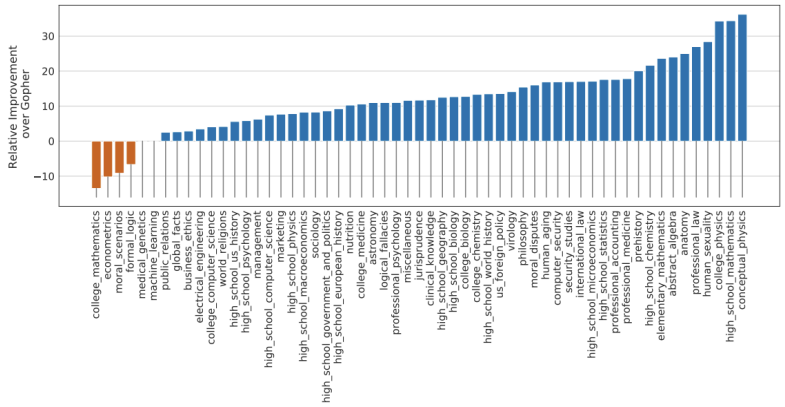

Figure 6 | **MMLU results compared to *Gopher*** We find that *Chinchilla* outperforms *Gopher* by 7.6% on average (see Table 6) in addition to performing better on 51/57 individual tasks, the same on 2/57, and worse on only 4/57 tasks.

References

- Neural Machine Translation
- Understanding Deep Learning
- A Neural Probabilistic Language Model
- DPO
- Build an LLM From Scratch
- RAG
- InstructGPT
- RLHF Book